Module: Natural Language Processing

PROJECT REPORT:

Mini search engine

(A document retrieval pipeline with lexical and semantic search models)

Author:
[Hmadna Aymane]

my github: [github.com/AymaneHmadna]

Academic Year: 2025-2026

Contents

1 Introduction:

The rapid growth of digital academic content makes manual document retrieval increasingly impractical. **mini search engine**, a complete nlp based document search system applied to a corpus of 249 academic pdfs spanning five core themes: Artificial intelligence, machine learning, data science, programming, and computer networks.

The project is organized into two main parts:

- **Part 1:** data collection, preprocessing, and text representation responsible for building the document index that the search engine operates on.
- **Part 2:** search engine implementation using the indices produced in part 1, comparing cosine similarity and BM25 algorithms.

2 Part 1:

2.1 Data sources:

The corpus was assembled from two complementary sources to ensure thematic and linguistic diversity:

1. **arXiv.org**: scientific papers retrieved via the arXiv api using topic specific queries . Each paper was downloaded in pdf format.
2. **University course pdfs**: French academic course materials covering programming, algorithms, and introductory AI, collected from open educational sources.

The collection script (`data.py`) queries the arXiv api and downloads up to 20 pdfs per topic:

Listing 1: arXiv topic queries used for data collection.

```
TOPICS = [
    "machine learning introduction",
    "deep learning neural networks",
    "artificial intelligence survey",
    "data science methodology",
    "computer networks protocol",
    "natural language processing",
    "python programming tutorial",
    "data preprocessing techniques",
    "information retrieval search",
    "reinforcement learning",
]
```

2.2 PDF text extraction:

Raw text was extracted from each pdf using `pdfplumber`, which handles multicolumn layouts and preserves text order better than basic pdf parsers:

Listing 2: PDF text extraction using pdfplumber (extract.py).

```
with pdfplumber.open(pdf_path) as pdf:
    for page in pdf.pages:
```

```
        extracted = page.extract_text()
        if extracted:
            text += extracted + "\n"
```

2.3 Text cleaning:

Before analysis, extracted text was cleaned using `clean.py`:

Listing 3: Text cleaning pipeline (clean.py).

```
def clean_text(text):
    text = re.sub(r'\bcid\d+\w*\b', ' ', text, flags=re.IGNORECASE)
    text = re.sub(r'http\S+|www\.\S+', ' ', text)
    text = re.sub(r'\S+@\S+', ' ', text)
    text = re.sub(r"\s+", " ", text)
    text = re.sub(r"[^a-zA-Z0-9àâäéèêëîïôùûüçÀÂÄÉÈÊËÎÏÔÙÛÜÇ\s]", "",
        text)
    words = text.split()
    words = [w for w in words if len(w) <= 25]
    text = text.strip()
    return text
```

Documents with fewer than 20 words after cleaning were discarded.

2.4 Dataset statistics:

Table 1: Final corpus statistics after extraction and cleaning.

Metric	Value
Total documents	249
French documents	61 (24.5%)
English documents	188 (75.5%)
Total words	≈5,500,000
Unique words	≈342,000
Average doc length	733 words
Longest document	187,988 words
Shortest document	20 words

Table 2: Thematic distribution of the corpus.

Theme	Lang.	Source
Machine learning & deep learning	EN/FR	arXiv + courses
Artificial intelligence	EN/FR	arXiv + courses
Data science	EN	arXiv
Programming	FR	University courses
Computer networks & protocols	EN	arXiv

2.5 Exploratory data analysis:

Three analyses were performed on the cleaned corpus using `eda.py`.

2.5.1 Document length distribution:

Figure 1 shows a heavily right skewed distribution. The mode is in the 0 to 5,000 word range with approximately 120 documents, consistent with a mixed corpus of short arXiv papers and long university course pdfs.

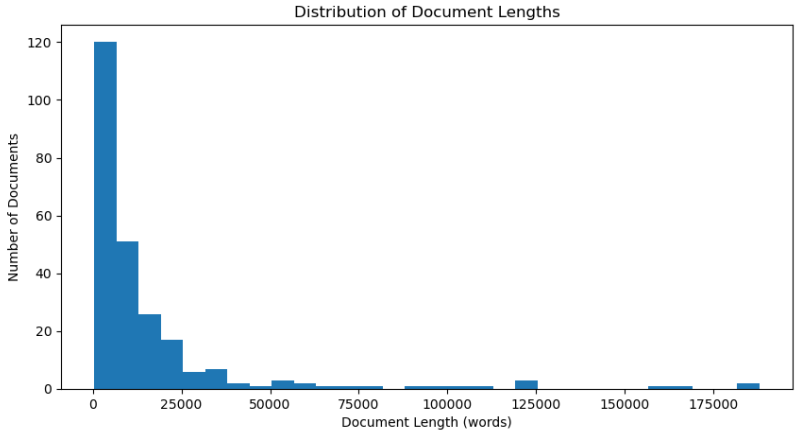


Figure 1: Distribution of document lengths. Most documents are under 5,000 words; a few long course pdfs extend up to 187,988 words. Three documents exceed 125,000 words.

2.5.2 Most frequent words before preprocessing:

Figure 2 shows the top 20 words before preprocessing. The most frequent words are: *de* (103k), *the* (91k), *la* (58k), *a* (54k), *of* (49k), *et* (43k), all stopwords from both french and english, demonstrating the necessity of stopword removal before any vectorization.

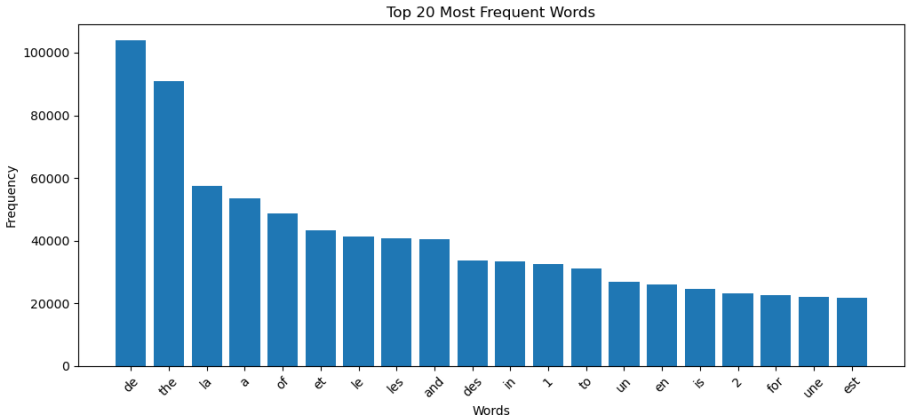


Figure 2: Top 20 most frequent words before preprocessing. Stopwords dominate, motivating their removal in Step 2.

2.5.3 Word cloud:

Figure 3 shows the corpus word cloud after basic cleaning. Thematic terms such as *data*, *model*, *learning*, *network*, and *algorithm* are prominent, confirming alignment with the five target themes.

Figure 3: Word cloud of the full corpus. Key thematic terms confirm topical alignment.

2.6 Pipeline overview:

The preprocessing pipeline is used in the project in `processed.py` and consists of four sequential steps applied uniformly to all 249 documents:

$$\text{Raw text} \xrightarrow{\text{Tokenize}} \text{Tokens} \xrightarrow{\text{Filter}} \text{Clean tokens} \xrightarrow{\text{Stop removal}} \text{Content tokens} \xrightarrow{\text{Stem}} \text{Stems}$$

2.7 Step by step implementation:

2.7.1 Tokenization

NLTK's `word_tokenize` is applied to the lowercased text. Unlike a simple `split()`, it correctly handles punctuation, contractions, and accented french characters.

2.7.2 Stopword removal:

A combined bilingual stopword list is constructed:

Listing 4: Bilingual stopword construction (processed.py).

```
stop_words = set(stopwords.words("english"))
stop_words.update(stopwords.words("french"))
stop_words.update({
    "would", "could", "also", "one", "know", "get",
    "like", "thank", "im", "ive", "fig", "figure",
    "table", "al", "pp", "vol", "et", "ibid", "ref",
    "dun", "dune", "comm", "cett", "peut", "tout",
    "tous", "etre", "dont", "cela", "cette", "celui",
    "celle", "ceux", "quand", "bien", "alors", "ainsi",
    "donc", "mais", "avoir", "faire", "tres", "encore",
    "toute", "apres", "avant", "entre", "selon", "lors",
    "depuis", "nbsp", "amp", "quot",
})
```

The custom list addresses two categories of noise:

- **PDF artifacts:** tokens like `nbsp`, `amp`, `quot` that survive character-level cleaning.

- **Domain noise:** academic writing words (`al`, `ibid`, `fig`) that carry no thematic information.

2.7.3 Length filtering:

Words with $\text{len}(w) \leq 1$ or $\text{len}(w) \geq 25$ are removed. One character tokens are non informative; very long tokens are typically pdf concatenation artifacts not caught by the cleaning step.

2.7.4 Bilingual stemming:

Language is detected per document: if any french indicator word is present in the tokenized document, the french stemmer is applied to all its tokens; otherwise the english stemmer is used.

Listing 5: Document level language detection and stemming (processed.py).

```
FRENCH_WORDS = {"le","la","les","de","du","des","un","une","est","avec",
    "pour"}
def preprocess(text):
    words = word_tokenize(text.lower())
    is_french = any(w in FRENCH_WORDS for w in words)
    stemmer = stemmer_fr if is_french else stemmer_en
    clean_words = [
        w for w in words
        if w.isalpha() and w not in stop_words and 1 < len(w) < 25
    ]
    return [stemmer.stem(w) for w in clean_words]
```

2.8 Stemming vs lemmatization: Justified choice:

Why stemming? We chose **SnowballStemmer** over lemmatization for the following reasons:

Table 3: Stemming vs Lemmatization comparison for our use case.

Criterion	Stemming	Lemmatization
Speed	**Fast** rule-based suffix removal	Slow requires dictionary lookup and POS tagging
Bilingual support	**SnowballStemmer supports FR + EN natively**	NLTK WordNetLemmatizer is english only; french requires spaCy
Output readability	Reduced stems (*learn* → *learn*, *learning* → *learn*)	Readable base forms (*running* → *run*)
Corpus size dependency	**None** fully rule based	requires large dictionaries, less robust on technical vocabulary
Suitability for IR	**Standard in information retrieval** conflates morphological variants effectively	more suitable for NLG or tasks requiring grammatical correctness

Conclusion: For a bilingual document retrieval task over a technical academic corpus, stemming is the appropriate choice. It reduces morphological variants (*networks*, *networking* → *network*), requires no external dictionaries, and natively supports both french and english via `SnowballStemmer`.

2.9 Preprocessing output:

Table 4: Files produced by processed.py.

File	Content
`processed/filenames.pkl`	Ordered list of 249 filenames
`processed/processed_documents.pkl`	Preprocessed texts (string format)

Example document 0 before and after preprocessing:

```
# Before:
"Optimization Methods for Supervised Machine Learning From Linear
 Models to Deep Learning Frank E Curtis Katya Scheinberg 2017..."

# After preprocessing:
"optim method supervis machin learn linear model deep learn
 frank curti katya scheinberg abstract goal tutori introduc
 key model algorithm open question relat use optim method..."
```

The vocabulary is reduced from ≈342,000 raw tokens to ≈5,000 unique stems used by Word2Vec.

2.10 Overview and method selection:

Text representation transforms preprocessed token sequences into numerical vectors for similarity computation. We discuss two methods:

- **TF-IDF**:statistical, sparse representation.
- **Word2Vec**:neural, dense semantic representation.

2.11 Method 1: TF-IDF:

2.11.1 Mathematical formulation:

TF-IDF assigns each term t in document d a weight:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t) \tag{1}$$

$$\text{TF}(t, d) = \frac{f_{t,d}}{\sum_{t'} f_{t',d}} \qquad \text{IDF}(t) = \log\left(\frac{N}{1 + df_t}\right) + 1 \tag{2}$$

where $f_{t,d}$ is the frequency of term t in document d, N is the total number of documents, and df_t is the number of documents containing t.

2.11.2 Properties and limitations for our task:

Table 5: TF-IDF properties in the context of our retrieval task.

Property	Impact
Sparse representation	(249× ∼31,000) matrix high dimensional but only 4 non zero entries.
No semantic understanding	*"machine learning"* and *"apprentissage automatique"* are treated as completely different terms, even though they mean the same thing.
Exact term matching	A document mentioning *"Python"* once in a 500 page pdf may score similarly to a course entirely dedicated to python.
Robust on small corpora	does not require training; works equally well on 10 or 10,000 documents.

2.11.3 Why TF-IDF is not the final method:

The key limitation of TF-IDF for our use case is the **topic vs mention problem**: TF-IDF cannot distinguish whether a word is the *subject* of a document or merely mentioned in passing. A document about networks that mentions *"machine learning"* once may outrank a dedicated machine learning course if it is longer.

This motivates the use of Word2Vec, which captures the semantic context of words rather than their raw frequency.

2.12 Method 2: Word2Vec:

2.12.1 Principle:

Word2Vec learns dense vector representations by training a shallow neural network to predict context words given a target word (**Skip gram** mode). Words appearing in similar contexts receive similar vector representations, encoding semantic relationships.

A document vector is the **mean** of its word vectors:

$$\vec{d} = \frac{1}{|W_d|} \sum_{w \in W_d} \vec{w} \tag{3}$$

where W_d is the set of vocabulary words present in document d.

2.12.2 Implementation:

Listing 6: Word2Vec training (representation.py).

```
from gensim.models import Word2Vec
sentences = [doc.split() for doc in processed_documents]
w2v_model = Word2Vec(
    sentences,
    vector_size=100,
    window=5,
```

```
    min_count=2,
    workers=4,
)
```

Listing 7: Document vectorization (representation.py).

```
def get_document_vector(doc_tokens, model, vector_size=100):
    valid_words = [word for word in doc_tokens if word in model.wv]
    if not valid_words:
        return np.zeros(vector_size)
    word_vectors = np.array([model.wv[word] for word in valid_words])
    return word_vectors.mean(axis=0)
```

2.12.3 Hyperparameter choices:

Table 6: Word2Vec hyperparameters and justifications.

Parameter	Value	Justification
`vector_size`	100	Sufficient for a corpus of 249 documents; larger dimensions would risk overfitting.
`window`	5	Standard value; captures local context within academic sentences.
`min_count`	2	Removes hapax legomena (single-occurrence tokens) that add noise.

2.12.4 Output files:

Table 7: Files produced by representation.py.

File	Content
`processed/word2vec.model`	trained Word2Vec model
`processed/w2v_matrix.pkl`	document matrix

2.13 TF-IDF vs Word2Vec:

Table 8: Qualitative comparison of TF-IDF and Word2Vec.

Criterion	**TF-IDF**	**Word2Vec**
Representation	Sparse	Dense
Semantic understanding	No frequency based only	Yes contextual similarity
Handles synonyms	No *car* $\neq$ *automobile*	Yes similar vectors
Corpus size requirement	None robust on small data	Prefers large corpora
Bilingual support	Direct independent terms	Limited 61 FR docs insufficient
Topic vs mention	Cannot distinguish	Better separation via context
Computation	Instant	Requires training
Interpretability	High weights are readable	Low dense vector space
Implementation	Theoretical in this part	**Implemented** (`representation.py`)

2.14 Justified final choice: Word2Vec:

Word2Vec is selected as the used in the project representation method for the following reasons:

1. **Semantic retrieval:** Word2Vec captures semantic proximity between terms. The query *"how machines understand humain language"* can match a document about *"nlp"* even without exact word overlap and even with mixed languages or errors "humain".

2. **Topic vs mention:** The mean document vector is dominated by the most frequent semantic directions in the document, naturally down weighting incidental mentions.

3. **Robustness to typos:** The search engine uses Word2Vec vectors with cosine similarity, words with similar stems map to nearby vectors even with spelling variation.

3 Part 2:

3.1 Algorithm Construction and Mathematics:

To perform our searches, we contrasted an unsupervised Machine Learning model with a probabilistic reference baseline.

3.2 Semantic approach and ml: Word2Vec + cosine similarity:

The first method relies on the embeddings generated by **Word2Vec** (dimension $d = 100$). When a user query Q is submitted, the system calculates its representative vector $\mathbf{v}_Q$ as the mean of the vectors of its valid words:

$$\mathbf{v}_Q = \frac{1}{|Q|} \sum_{w \in Q} \mathbf{v}_w \tag{4}$$

To extract relevant documents, we use **Cosine similarity**. This metric calculates the cosine of the angle between the query vector $\mathbf{v}_Q$ and the vector of each document $\mathbf{d}_i$ in the 100-dimensional space:

$$\text{Score}_{\text{Cosine}}(Q, d_i) = \frac{\mathbf{v}_Q \cdot \mathbf{d}_i}{\|\mathbf{v}_Q\| \|\mathbf{d}_i\|} = \frac{\sum_{j=1}^{d} v_{Q,j} \cdot d_{i,j}}{\sqrt{\sum_{j=1}^{d} v_{Q,j}^2} \sqrt{\sum_{j=1}^{d} d_{i,j}^2}} \tag{5}$$

This process corresponds to a **K nearest neighbors** classification algorithm, which extracts the $K = 5$ documents maximizing this score.

3.3 The Lexical baseline: The BM25 algorithm:

In order to compare our artificial intelligence model to a standard mathematical formula, we used in the project **BM25**, an advanced evolution of TF-IDF. The score of a document d_i for a query Q is defined by the following equation:

$$\text{Score}_{\text{BM25}}(Q, d_i) = \sum_{q \in Q} \text{IDF}(q) \cdot \frac{f(q, d_i) \cdot (k_1 + 1)}{f(q, d_i) + k_1 \cdot \left(1 - b + b \cdot \frac{|d_i|}{\text{avgdl}}\right)} \tag{6}$$

Unlike TF-IDF, BM25 introduces:

- Term frequency saturation ($k_1 = 1.5$): a word repeated a large number of times stops exponentially increasing the score.
- A length penalty ($b = 0.75$) related to the document length $|d_i|$ compared to the corpus average avgdl.
- The inverse document frequency $\text{IDF}(q) = \ln\left(\frac{N - n(q) + 0.5}{n(q) + 0.5} + 1\right)$.

3.4 Mathematical evaluation:

To mathematically evaluate precision, recall, and f1 score, we coded a script (`evaluate.py`) defining a ground truth on 5 complex queries related to the field of *reinforcement learning*.

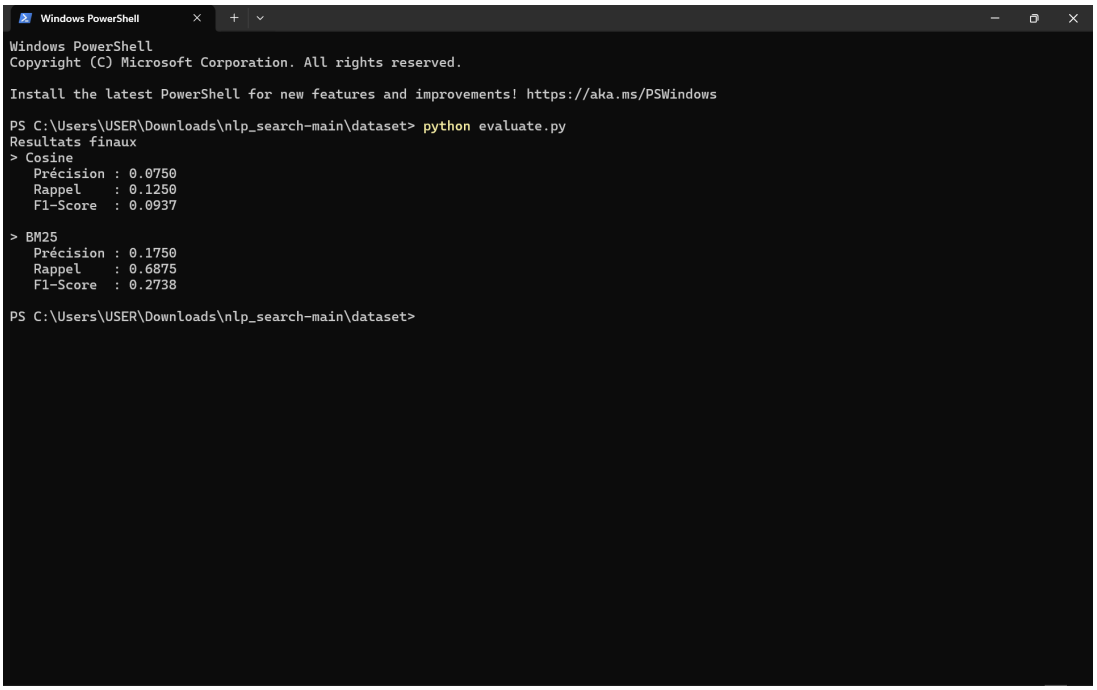


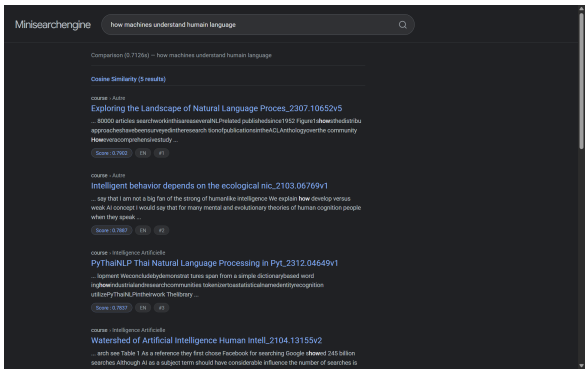
Figure 4: Results of the automated model evaluation

The evaluation shows that BM25 outperforms Word2Vec with a 0.27 f1 score, as its probabilistic approach excels at isolating specific technical terms within a small corpus. While Word2Vec successfully retrieves relevant content, its 0.07 precision indicates that vector based proximity often includes irrelevant documents, highlighting the need for hybrid search architectures.

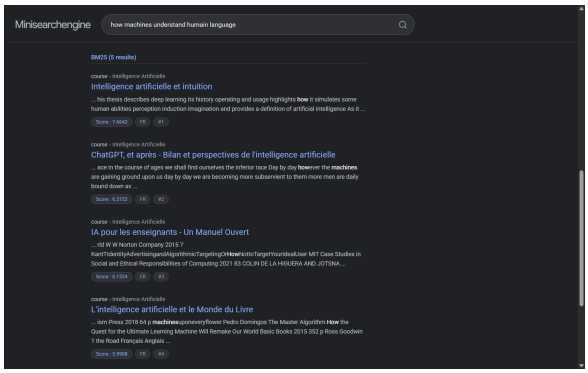
3.5 User interface and demonstration:

The entire engine was deployed using the **flask** framework. The interface dynamically generates a text excerpt centered on the keywords found in the pdf.

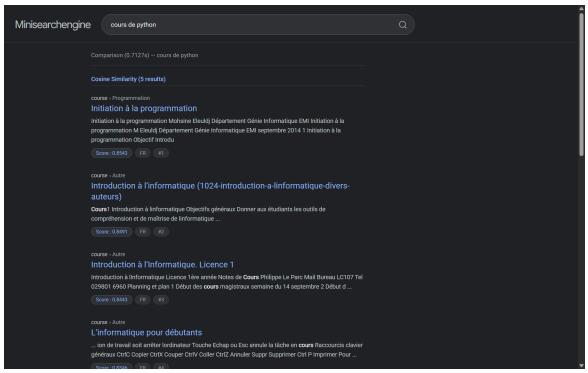
Figure 5: Search interface displaying



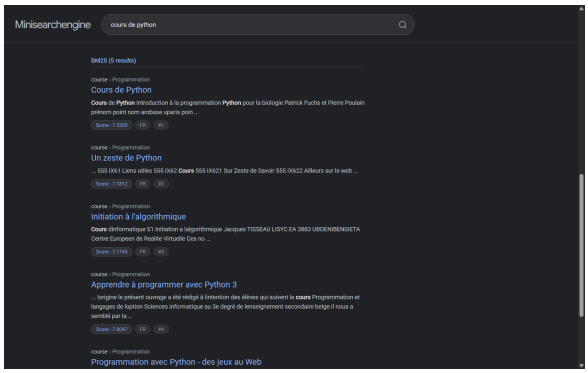
(a) Req 1: Cosine (Word2Vec)



(b) Req 1: BM25



(c) Req 2: Cosine (Word2Vec)



(d) Req 2: BM25

Figure 6: Comparison of algorithms on two different queries

3.6 Analysis, discussion, and perspectives:

These results allow us to draw clear conclusions about our architecture:

3.7 Discussion of results:

1. **The impact of IDF on technical words:** BM25 is good on our dataset. Mathematically, the IDF gives a massive weight to rare terms (*"Pommerman", "Maclaurin"*). BM25 therefore instantly filters the exact documents without "noise".

2. **The curse of dimensionality for Word2Vec:** The Word2Vec score suffers from a low Precision (0.07). A vector model will always seek to return K neighbors, even if the "closest" is actually semantically distant. The relevant answer is therefore drowned under mathematical false positives. Furthermore, a corpus of 249 documents does not provide enough co-occurrences to finely separate concepts in a 100 dimensional space.

3.8 Architectural evolution perspectives:

Faced with these limitations, several engineering perspectives emerge to improve the system:

- **Hybrid search:** The optimal solution for production would be to implement a meta algorithm calculating $Score_{Final} = \alpha \cdot Score_{BM25} + (1 - \alpha) \cdot Score_{W2V}$ after normalization. This would force the document to possess both the exact words and the correct semantic field.

- **Pretrained models:** To overcome the small size of the corpus, replacing the locally trained Word2Vec with embeddings pretrained on Wikipedia fasttext or glove would offer the engine a better understanding of the general scientific vocabulary from the start.

4 Conclusion:

This project allowed for the construction of a complete nlp pipeline. The evaluation demonstrated a fundamental rule in data engineering: for a small, highly technical corpus, a classic probabilistic approach offers a better return on precision investment than an isolated neural network. The natural evolution of the *mini search engine* would turn towards a hybridization of the two methods to combine robustness and semantic understanding.